



Apache Spark

Sherif Ashraf

Data Management specialist @ Vodafone

Agenda

- Introduction
- History of Spark
- Architecture
- Deployment modes
- Transformations



Introduction



- Apache Spark is a unified computing engine and a set of libraries for parallel data processing on computer clusters.
- Spark supports multiple widely used programming languages (Python, Java, Scala, and R), includes libraries for diverse tasks ranging from SQL to streaming and machine learning.
- runs anywhere from a laptop to a cluster of thousands of servers. This makes it an easy system to start with and scale-up to big data processing or incredibly large scale.

Introduction

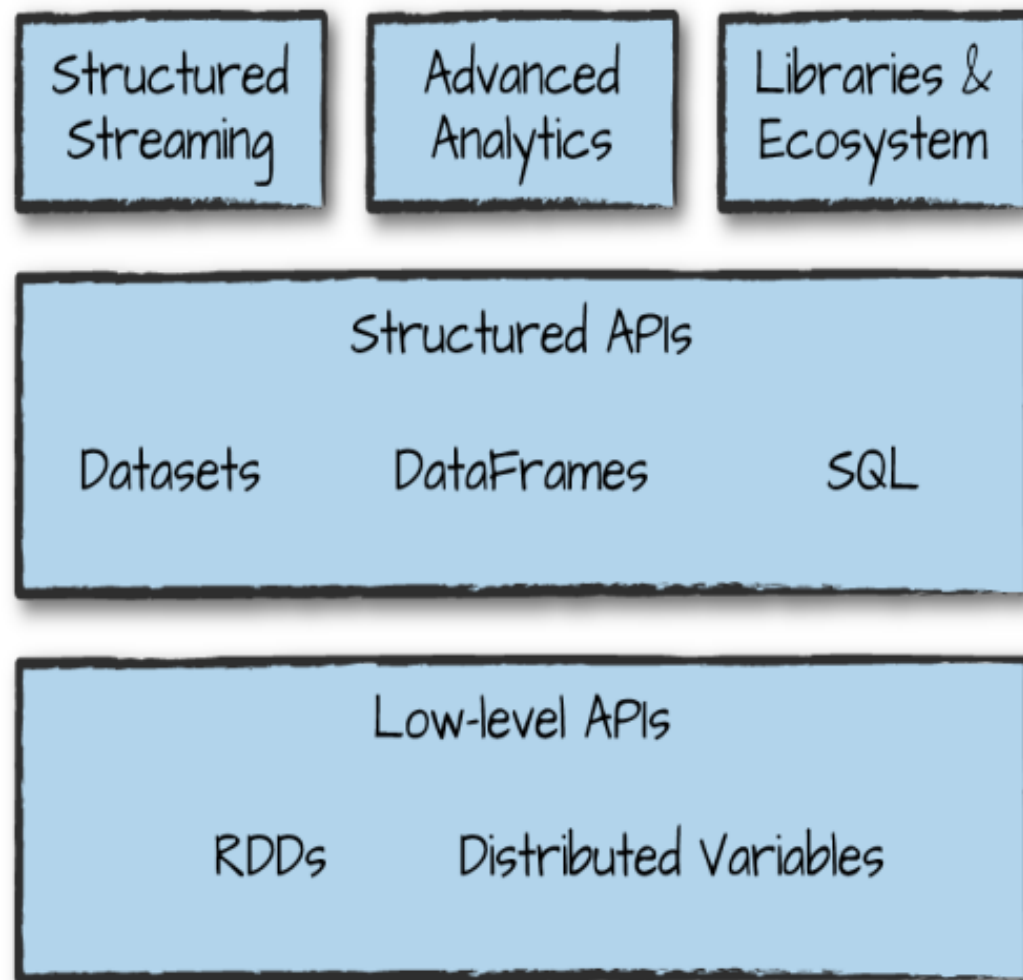


Figure 1-1. Spark's toolkit



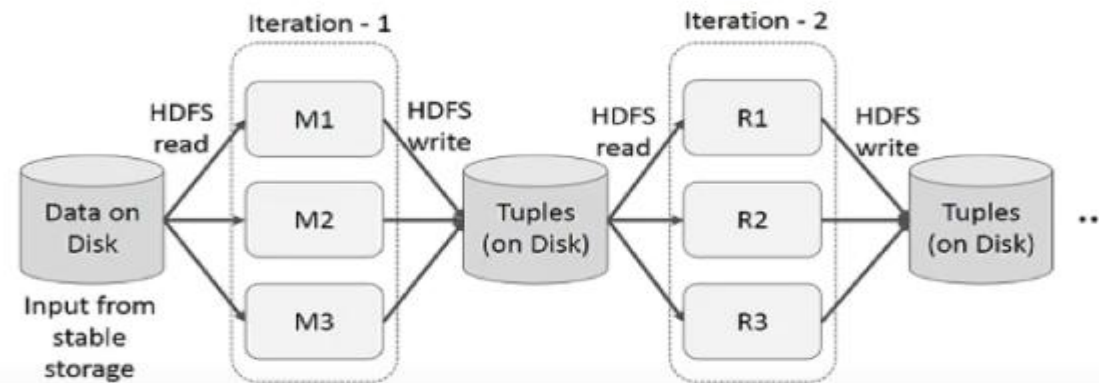
The Big Data Problem



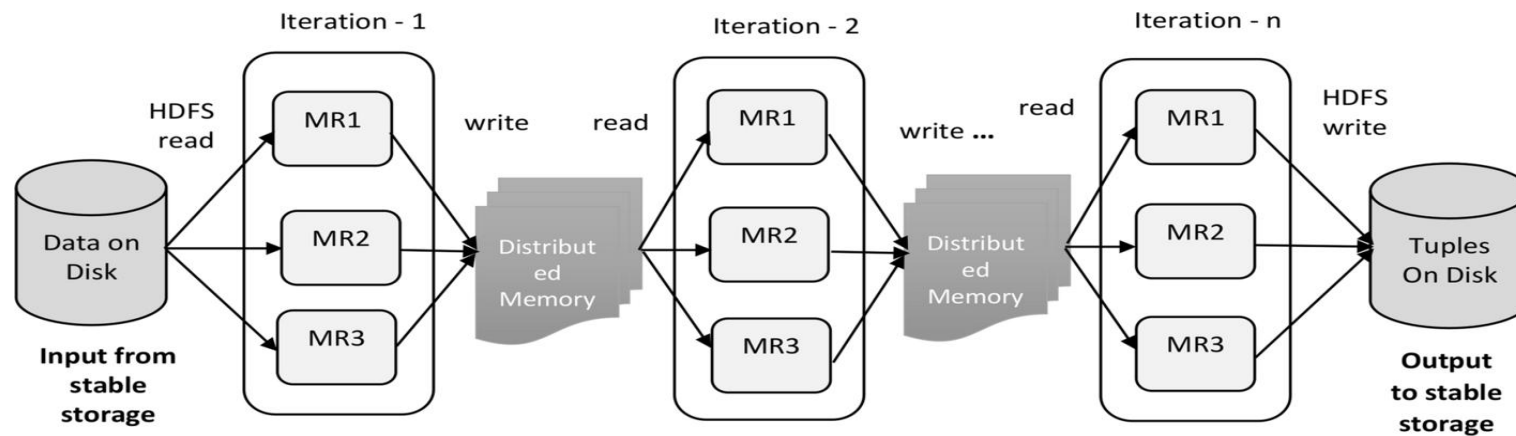
The history of Spark

Map Reduce and Spark

Map Reduce



Spark



Spark's Architecture



- Spark driver.
- SparkSession.
- Cluster manager.
- Spark executor.

Spark's Architecture

- **Spark driver.**
 - Responsible for instantiating a SparkSession.
 - Communicates with the cluster manager, it requests resources (CPU, memory, etc.) from the cluster manager for Spark's executors.
 - Once the resources are allocated, it communicates directly with the executors.
 - Transforms all the Spark operations into DAG computations, schedules them, and distributes their execution as tasks across the Spark executors.

Spark's Architecture

- **SparkSession**
 - SparkSession provides a single unified entry point to all of Spark's functionality.
 - Not only did it subsume previous entry points to Spark like the SparkContext, SQLContext, HiveContext, SparkConf, and StreamingContext, but it also made working with Spark simpler and easier.

Spark's Architecture



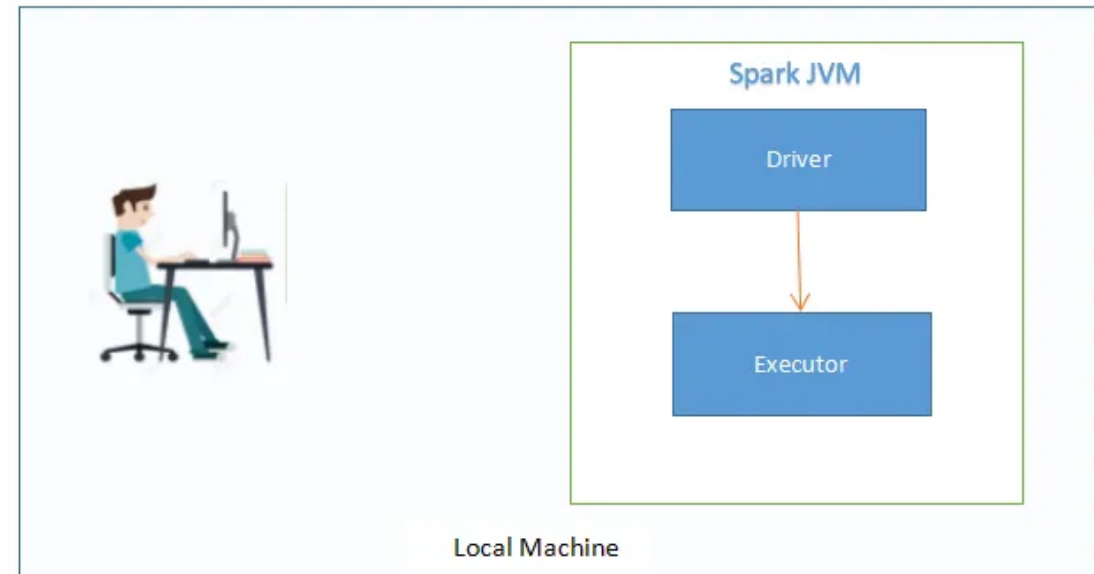
- **Cluster manager**
 - The cluster manager is responsible for managing and allocating resources for the cluster of nodes on which your Spark application runs.
 - Spark supports four cluster managers:
 - the built-in standalone cluster manager
 - Apache Hadoop YARN
 - Apache Mesos
 - Kubernetes.

Spark's Architecture

- **Spark executor**
 - A Spark executor runs on each worker node in the cluster.
 - The executors communicate with the driver program and are responsible for executing tasks on the workers.
 - In most deployments' modes, only a single executor runs per node.

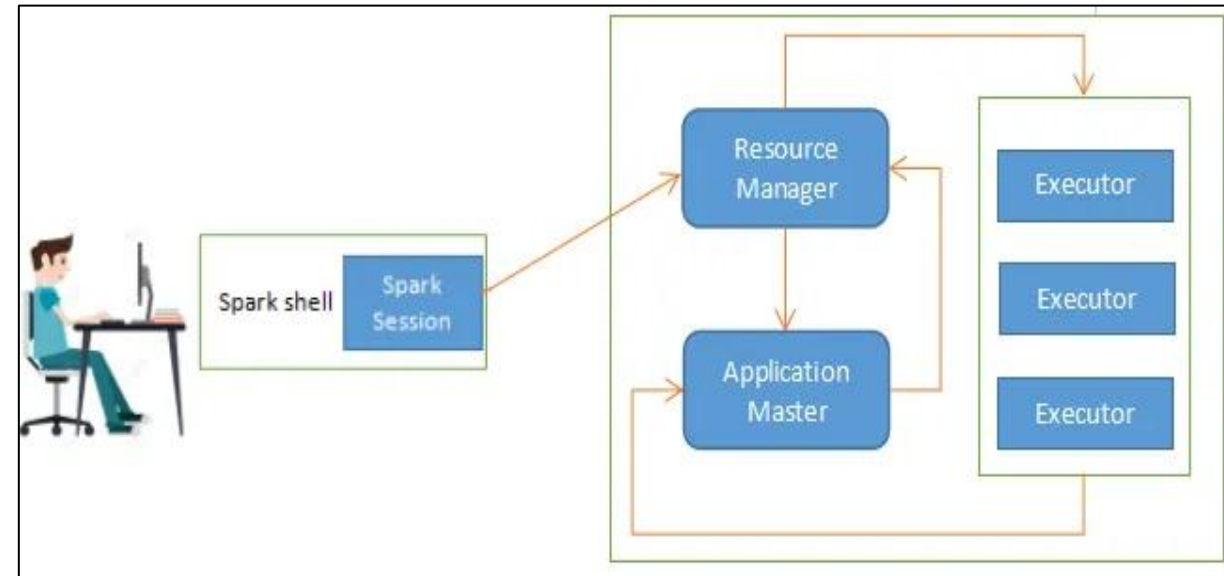
Deployment modes

- **Local Mode:**
 - In local mode, Spark runs on a single machine, using n cores of the machine. It is the simplest mode of deployment and is mostly used for testing and debugging.



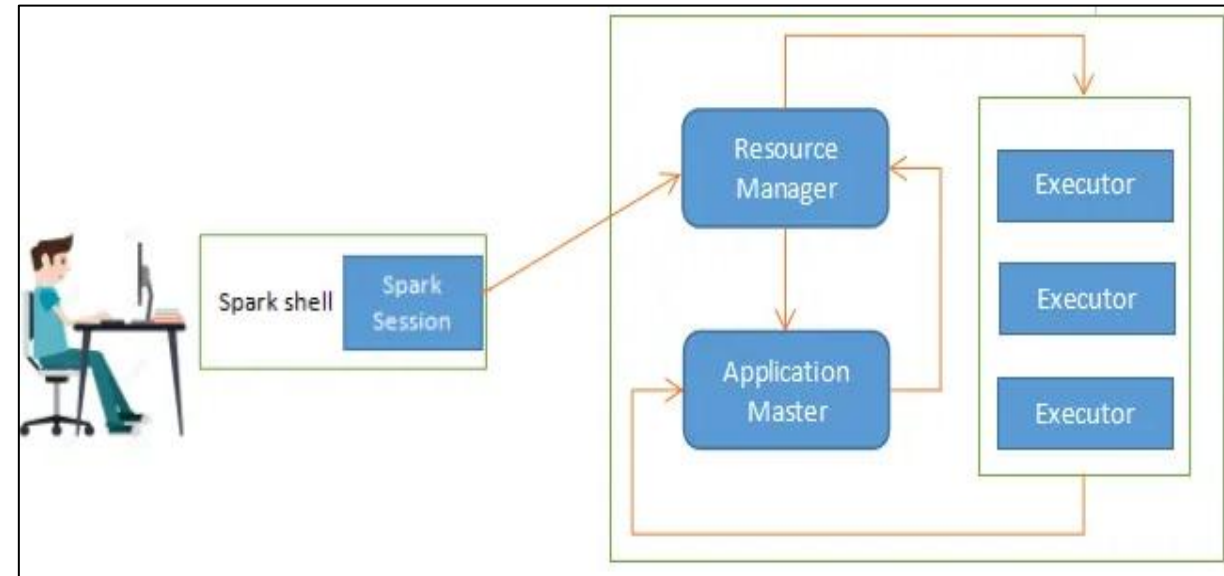
Deployment modes

- **YARN Client Mode:**
 - When you start a spark shell, application driver creates the spark session in your local machine which request to Resource Manager present in cluster to create Yarn application.
 - YARN Resource Manager start an Application Master (AM container). For client mode Application Master acts as the Executor launcher.
 - Application Master will reach to Resource Manager and request for further containers. Resource manager will allocate new containers.



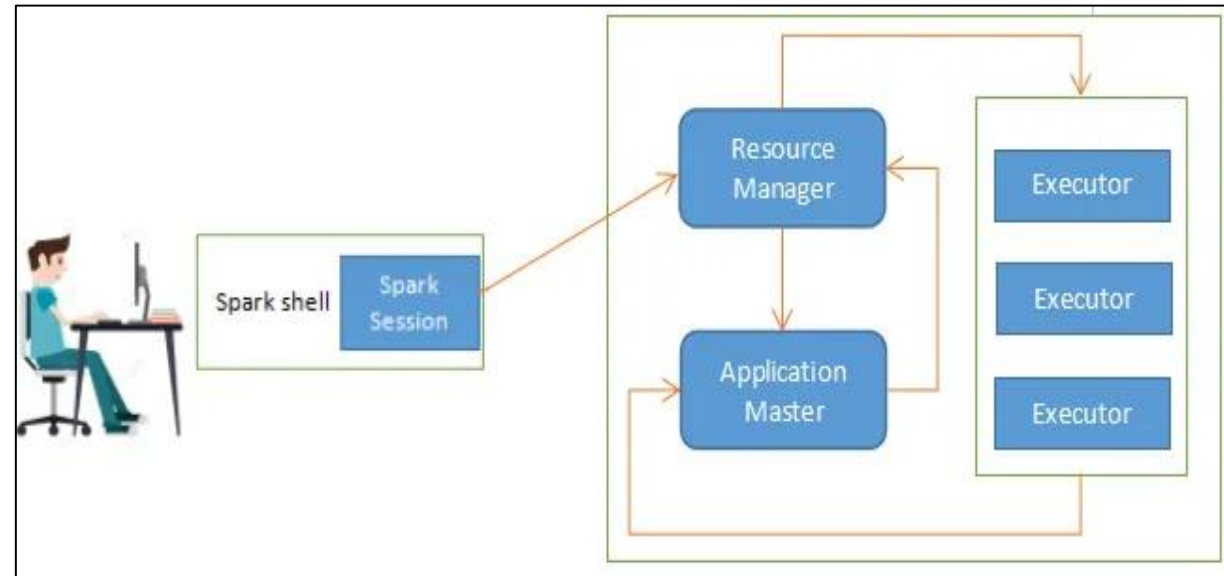
Deployment modes

- **YARN Client Mode:**
 - Each executor runs in its own JVM and can run multiple tasks concurrently.
 - these executors will directly communicate with Drivers which is present in the system in which you have submitted the spark application.
 - In this mode client have more control over the application. This enables user interaction with the application while it is running, progress monitoring, and application modification as needed. Additionally, it gives the user direct access from the client machine to the application's output and logs, which can be helpful for debugging and troubleshooting.



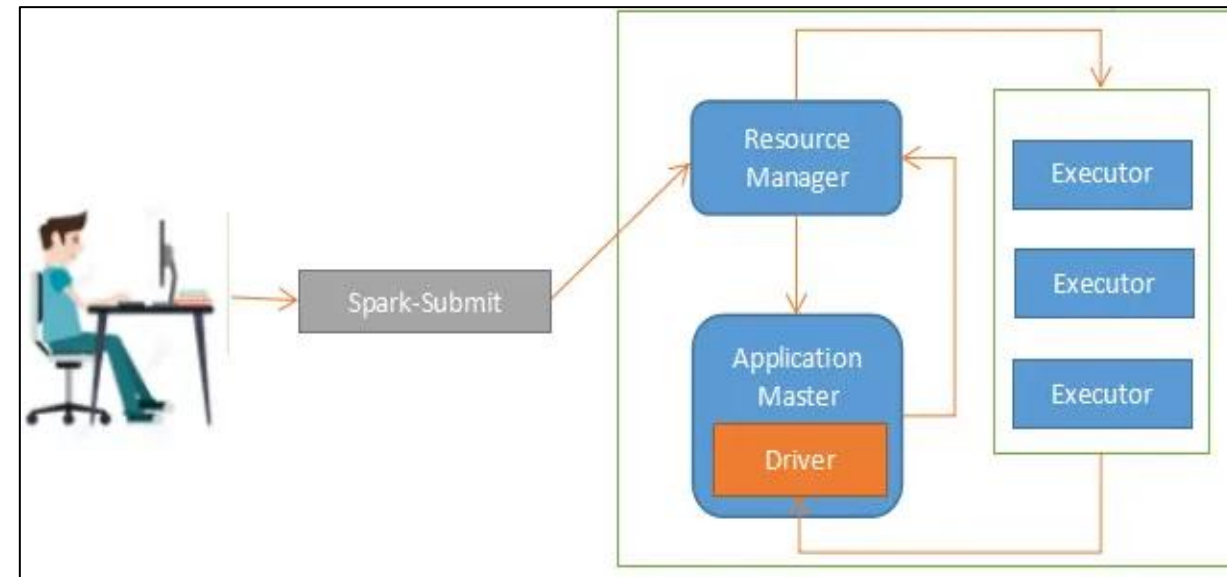
Deployment modes

- **YARN Client Mode:**
 - The major disadvantage of client mode is the driver node is running in client machine and therefore if we shutdown the machine or any issues occur, it will affect the entire application run.
 - Client mode is not suitable for Production. For debugging purpose, we can use this client mode.



Deployment modes

- **YARN Cluster Mode:**
 - Here Application Master will create driver in it and driver will reach to Resource Manager.
 - Client mode is suitable for Production. Or when submitting your spark application from a machine far from the cluster.
 - Less network latency between the driver and the executors.

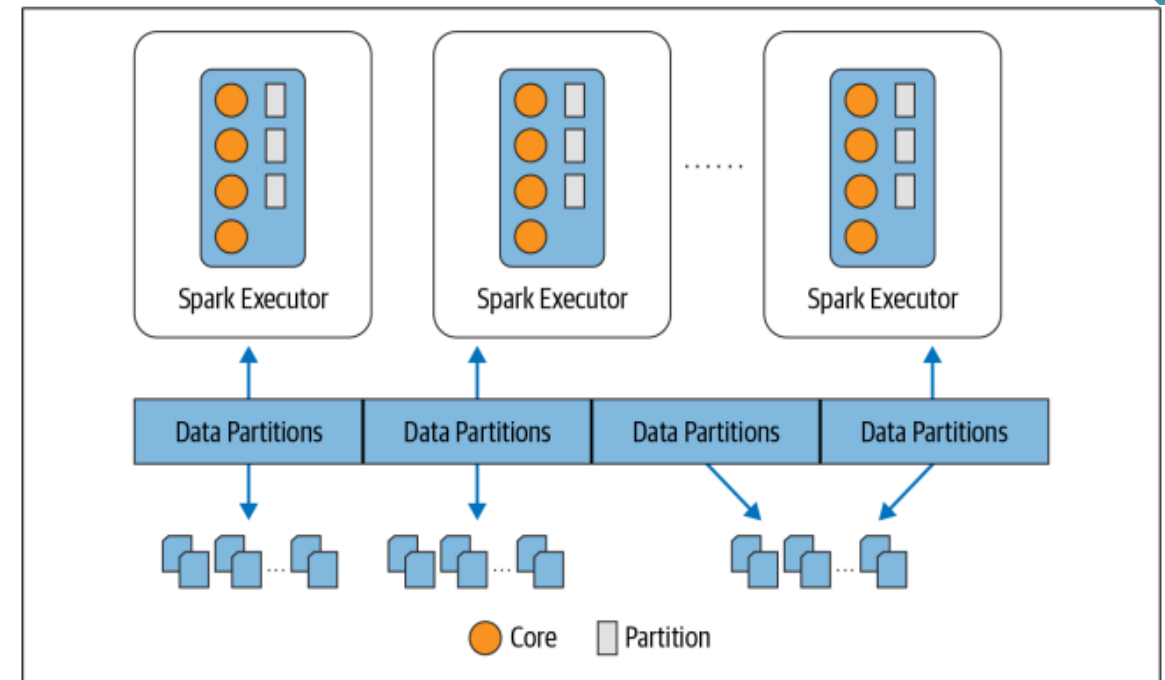


Deployment modes

Mode	Spark driver	Spark executor	Cluster manager
Local	Runs on a single JVM, like a laptop or single node	Runs on the same JVM as the driver	Runs on the same host
Standalone	Can run on any node in the cluster	Each node in the cluster will launch its own executor JVM	Can be allocated arbitrarily to any host in the cluster
YARN (client)	Runs on a client, not part of the cluster	YARN's NodeManager's container	YARN's Resource Manager works with YARN's Application Master to allocate the containers on NodeManagers for executors
YARN (cluster)	Runs with the YARN Application Master	Same as YARN client mode	Same as YARN client mode

Distributed data and partitions

- Spark executor is preferably allocated a task that requires it to read the partition closest to it in the network, observing data locality.
- Partitioning allows for efficient parallelism. A distributed scheme of breaking up data into chunks or partitions allows Spark executors to process only data that is close to them, minimizing network bandwidth. That is, each executor's core is assigned its own data partition to work on.



Transformations, Actions, and Lazy Evaluation

- **Transformations:**

- transform a Spark DataFrame into a new DataFrame without altering the original data.
- It will not change the original DataFrame; instead, it will return the transformed results of the operation as a new DataFrame.
- All transformations are evaluated lazily. That is, their results are not computed immediately, but they are recorded or remembered as a lineage.
- A recorded lineage allows Spark, at a later time in its execution plan, to rearrange certain transformations, coalesce them, or optimize transformations into stages for more efficient execution.
- Lazy evaluation is Spark's strategy for delaying execution until an action is invoked or data is "touched" (read from or written to disk).

- **Actions:**

- Operation that require return of a result on the console or writing an output in a file, database,... etc.
- An action triggers the lazy evaluation of all the recorded transformations.

Transformations	Actions
orderBy()	show()
groupBy()	take()
filter()	count()
select()	collect()
join()	save()

Narrow and Wide Transformations

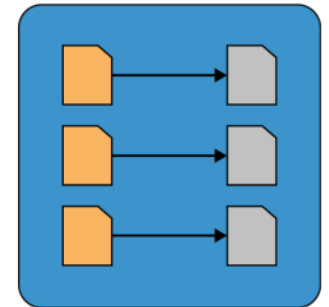
- **Narrow Transformations:**

- Any transformation where a single output partition can be computed from a single input partition is a narrow transformation.
- For example, in the previous code snippet, `filter()` and `contains()` represent narrow transformations because they can operate on a single partition and produce the resulting output partition without any exchange of data.

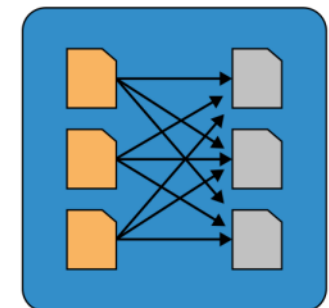
- **Wide Transformations:**

- Transformations that require exchange of data between partitions (shuffle).
- `groupBy()` or `orderBy()` instruct Spark to perform wide transformations, where data from other partitions is read in, combined, and written to disk.

Narrow Dependencies



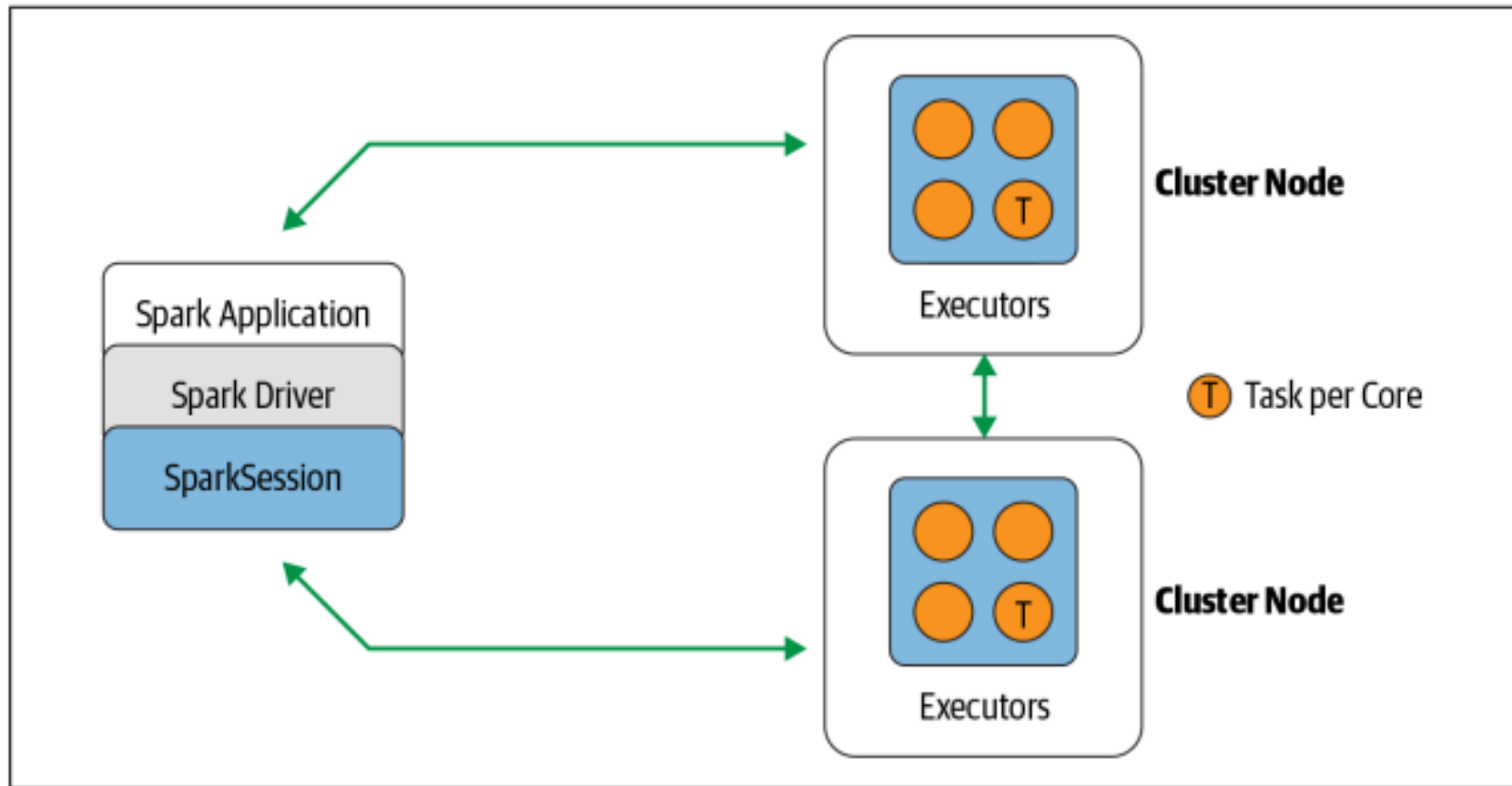
Wide Dependencies



Spark Application Concepts

- **Application** : A user program built on Spark using its APIs. It consists of a driver program and executors on the cluster.
- **SparkSession**: An object that provides a point of entry to interact with underlying Spark functionality and allows programming Spark with its APIs.
- **Job**: A parallel computation consisting of multiple tasks that gets spawned in response to a Spark action (e.g., `save()`, `collect()`).
- **Stage**: Each job gets divided into smaller sets of tasks called stages that depend on each other
- **Task**: A single unit of work or execution that will be sent to a Spark executor.

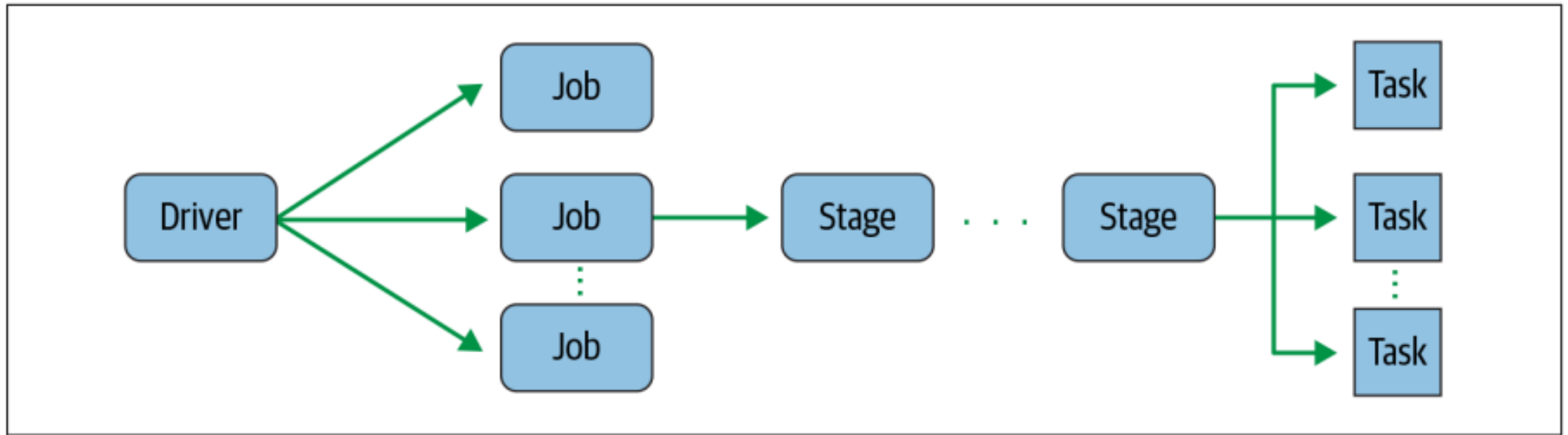
Spark Application Concepts



Spark Application

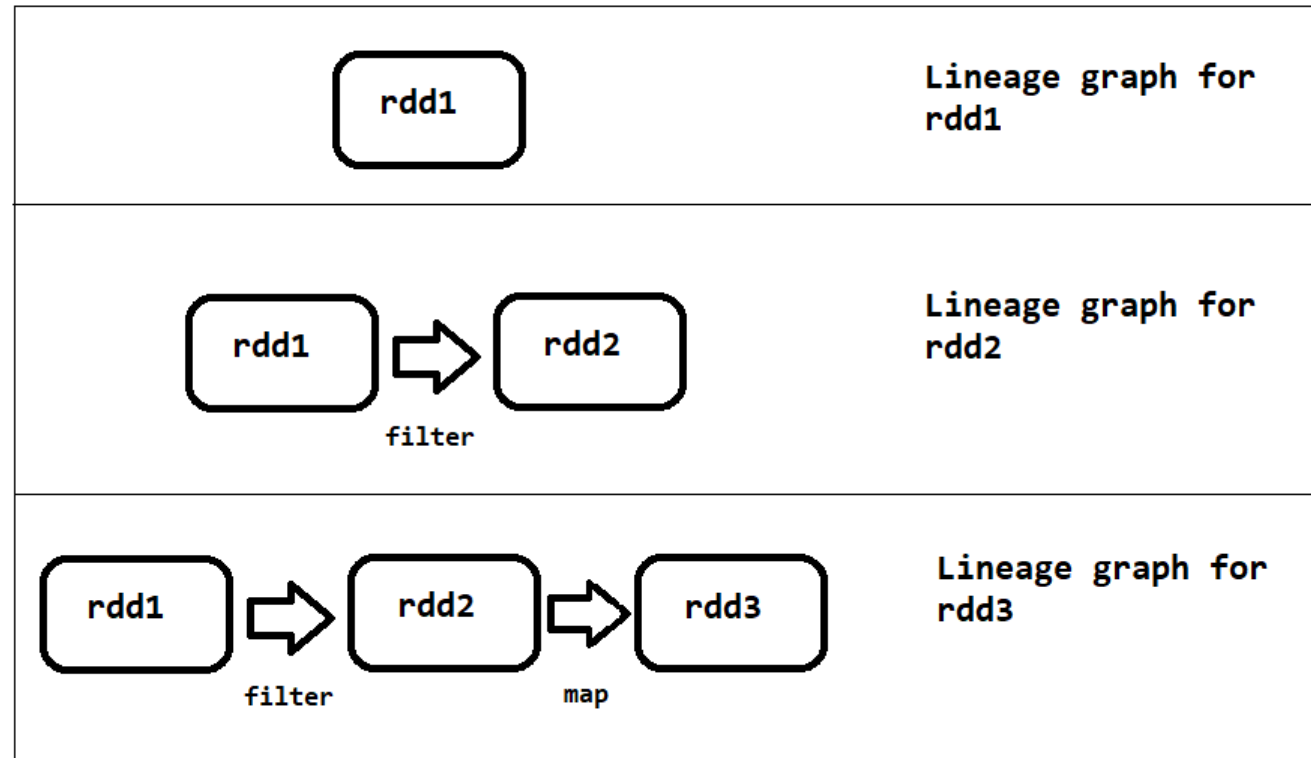
- **Spark Jobs** : the driver converts your Spark application into one or more Spark jobs.
 - It then transforms each job into a DAG. This, in essence, is Spark's execution plan, where each node within a DAG could be a single or multiple Spark stages.
- **Spark Stages**: stages are created based on what operations can be performed serially or in parallel.
 - Not all Spark operations can happen in a single stage, so they may be divided into multiple stages. (shuffle)
- **Spark Tasks**: Each stage is comprised of Spark tasks (a unit of execution), which are then federated across each Spark executor; each task maps to a single core and works on a single partition of data.
 - As such, an executor with 16 cores can have 16 or more tasks working on 16 or more partitions in parallel, making the execution of Spark's tasks exceedingly parallel.

Spark Application



RDD

- RDD is the main data structure for spark.
- RDD characteristics:
 - Resilient.
 - Distributed.
 - Immutable
 - Lazy evaluated.
- RDDs provide **transformations**, which evaluate lazily, and **actions**, which evaluate eagerly.



RDD Transformations and actions

- **Transformations:**

- Map
- Filter
- flatMap
- mapPartitions
- Union
- Intersection
- Distinct
- Coalesce
- Repartition

- **Transformations: (for pairwise RDD)**

- groupByKey
- reduceByKey
- sortByKey
- join

- **Actions:**

- Reduce
- Collect
- Count
- First
- Take
- saveAsTextFile
- foreach

- **Actions: (for pairwise RDD)**

- countByKey

Cache, persist, checkpoint

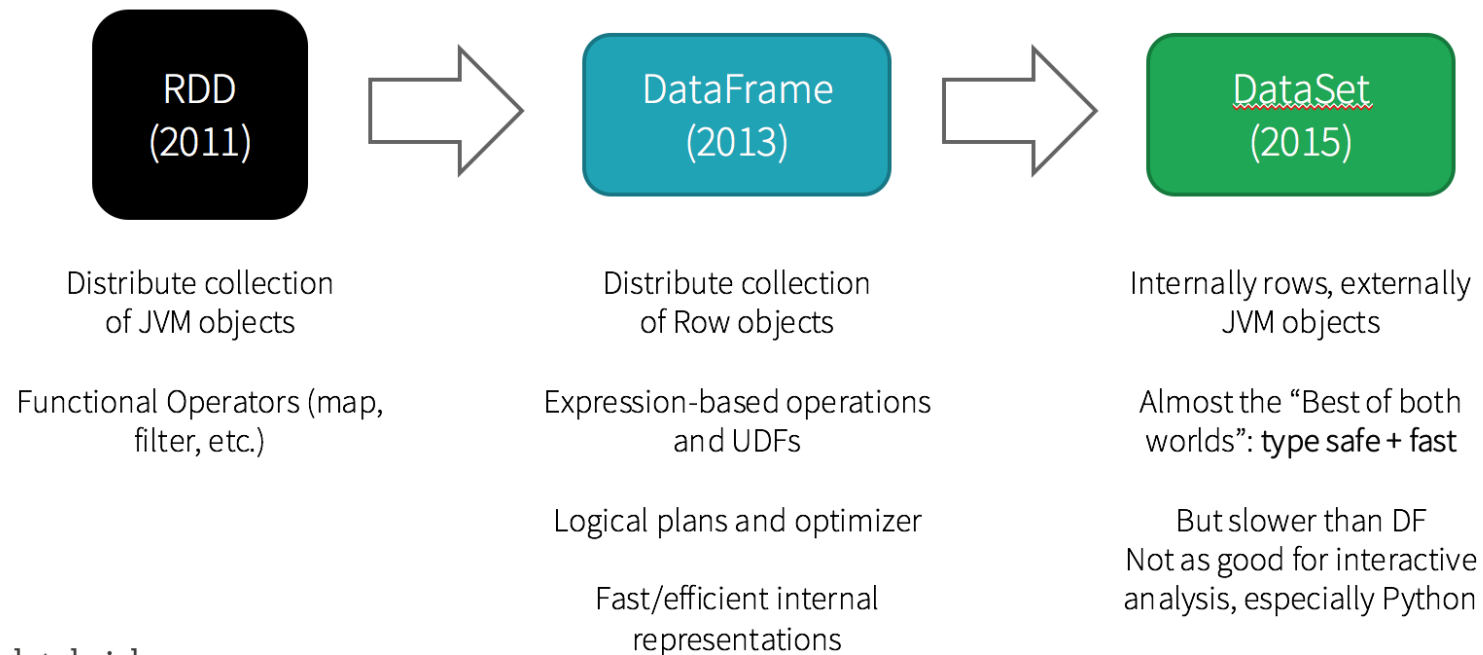
- **Cache():**
 - Caches the rdd or dataframe in memory to reuse it later on in your spark application for fast access.
- **Persist:**
 - Similar to cache but has storage levels, in-memory (like cache), in-memory and disk, and in disk only.
- **Checkpoint:**
 - Save the rdd or dataframe on specific directory on disk for future use in different spark application.

Repartition vs coalesce

- **Repartition :**
 - Used for increasing partitions count. Induces data shuffle.
- **Coalesce:**
 - Used for decreasing partitions count because it avoids shuffling, but may shuffle the data at lower chance.

DataFrame vs DataSet

Structured apis built on RDD.



DataFrame vs DataSet

RDDs

```
lang_rdd =  
  sc.parallelize  
    (("Python", 95),  
     ("R", 65),  
     ("Julia", 40))  
popular_langs =  
  lang_rdd.filter  
    lambda x1 x(1) >= 60  
print popular_langs  
collect()
```

DataFrames

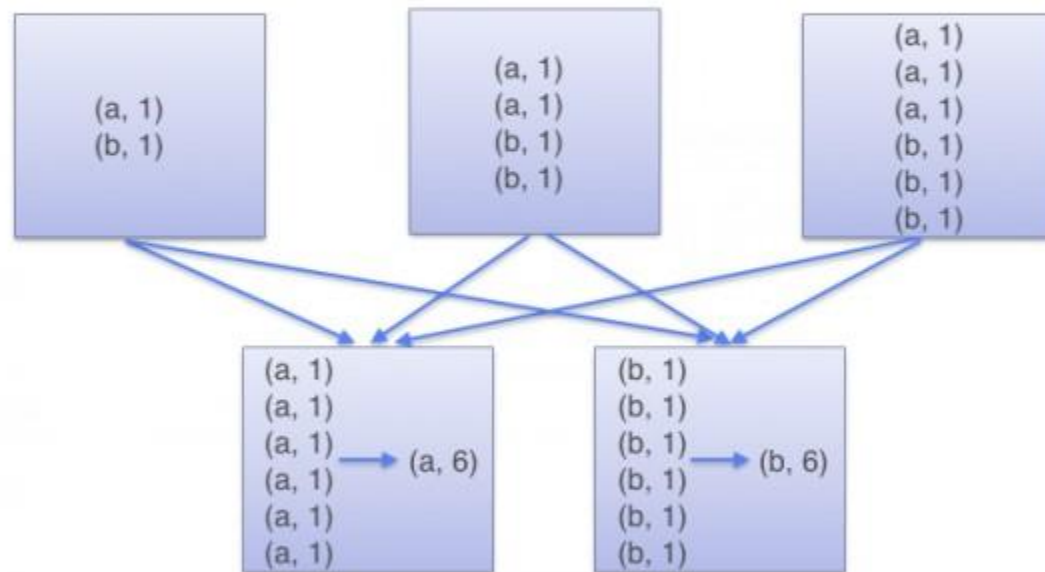
```
df = spark  
createDataFrame(  
  (("Python", 95),  
   ("R", 65),  
   ("Julia", 40)],  
  ('Language', 'Popularity')  
df.filter  
  df.Popularity > 60  
show
```

Datasets

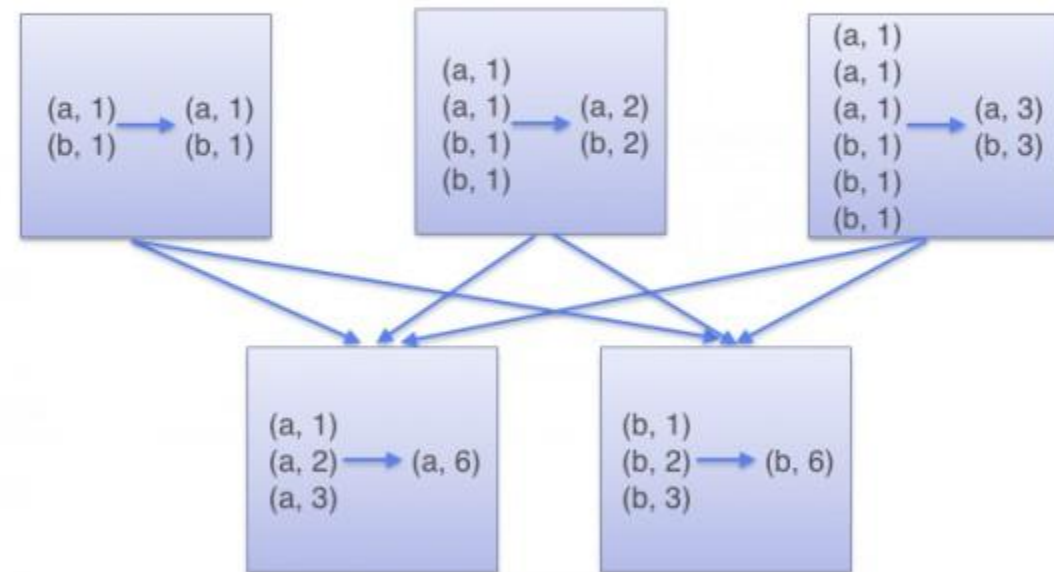
```
case class  
  Language(name: String,  
           popularity: Int)  
val ds =  
  Seq(Language("Python",  
               95), Language("R",  
                             65)).toDS()  
ds.filter(_popularity  
  > 60).show
```

GroupByKey vs ReduceByKey

GroupByKey



ReduceByKey



Thank you

